# ChronosDSE: Tail-Aware Design Space Exploration for GPGPUs


## Abstract

In the era of data center-scale computing, ensuring Service Level Agreements (SLAs) for Large Language Model (LLM) inference is paramount. Consequently, long-tail latency (e.g., P99) has superseded average throughput as the primary constraint for architectural evaluation. However, existing Design Space Exploration (DSE) frameworks fail to meet this requirement, as their reliance on static mean-based modeling masks the extreme volatility of modern workloads. LLM inference is driven by the rapid alternation of heterogeneous operators, causing microsecond-level bottleneck migrations between compute and memory subsystems. This dynamic behavior creates transient latency spikes that static metrics fundamentally fail to capture, leading to hardware designs that are efficient on average but fragile in worst-case scenarios. To bridge this gap, we propose ChronosDSE, the first framework to explicitly integrate long-tail performance metrics into the GPGPU DSE process. ChronosDSE shifts the modeling paradigm from static averaging to dynamic characterization, employing a novel *Chronos-Signature* to quantify bottleneck volatility and an adaptive sampling mechanism to preserve tail fidelity with minimal overhead. Experimental results demonstrate that ChronosDSE improves P99 performance by 10% compared to mean-based baselines, while reducing exploration effort by approximately 58% compared to exhaustive simulation. Crucially, its underlying methodology is architecture-agnostic, enabling seamless migration to diverse commercial GPU architectures beyond the validated platform.


## 1 INTRODUCTION

In the era of data center-scale computing, optimizing GPUs for Large Language Model (LLM) inference [13, 25] is crucial. Since user requests traverse multiple GPU nodes, service quality is dictated by long-tail latency (e.g., P99 [11]) rather than average throughput. A momentary stall in a single node can trigger cascading delays, violating strict Service Level Agreements (SLAs). Consequently, the primary objective of next-generation GPU architecture is to minimize these tail latencies to ensure robust system reliability.

To achieve this tail-latency robustness under power and area constraints, architects must perform rigorous Design Space Exploration (DSE [17, 19]). This process involves navigating billions of possible hardware configurations—ranging from cache hierarchies to compute unit allocations—to identify the optimal design. However, the effectiveness of DSE relies fundamentally on the fidelity of its performance modeling. If the underlying evaluation methodology fails to capture the transient latency spikes that occur in real-world scenarios, the DSE framework will inevitably converge on suboptimal architectures. These designs may appear efficient in simulation but will suffer from catastrophic performance degradation when deployed in actual production environments.

However, current DSE frameworks suffer from a fundamental disconnect: they rely on static mean-based metrics, whereas LLM workloads exhibit highly dynamic execution patterns. Driven by heterogeneous operators, hardware bottlenecks in LLMs migrate rapidly between compute and memory subsystems at a microsecond scale [1, 8]. Mean-based tools average out these high-frequency spikes, creating a false sense of stability—exemplified by the SOTA GCoM tool [15], which incurs over 125% error in LLM scenarios [3]. This fidelity gap forces architects to compromise between simulation speed and accuracy, leaving them unable to effectively optimize for tail latency.

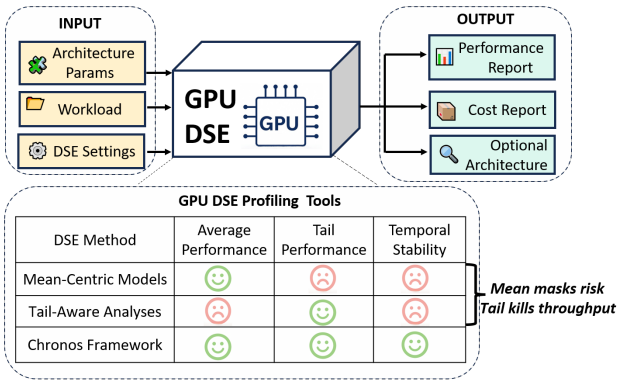



**Figure 1: An overview comparing ChronosDSE against existing DSE methods.**

Figure. 1 illustrates this methodological dilemma. As depicted in the comparison, existing mean-centric models effectively mask the risks of transient latency spikes (high average throughput but severe tail latency). Conversely, naive tail-aware approaches (e.g., exhaustive full-trace simulation) tend to fixate on capturing every transient detail, resulting in prohibitive simulation costs that make iterative DSE infeasible. ChronosDSE is designed to bridge this gap, simultaneously satisfying the requirements for average performance, tail robustness, and temporal stability.

To address this challenge and enable tail-aware architecture design, we propose ChronosDSE. To the best of our knowledge, this is the first framework to explicitly integrate long-tail performance metrics into the GPGPU DSE process without compromising exploration efficiency. Unlike prior works that ignore the temporal dimension, ChronosDSE fundamentally shifts the modeling paradigm from static averaging to dynamic characterization. To achieve this, we introduce three key innovations:

- Chronos-Signature for Dynamic Characterization: We introduce a novel signature mechanism to explicitly characterize the amplitude and frequency of bottleneck migration. While amplitude quantifies the severity of resource contention to aid in bottleneck identification, frequency captures the temporal instability, serving as a key driver for optimization.
- Adaptive Sampling for Tail Accuracy: We employ an adaptive sampling window mechanism that captures transient performance dips, ensuring that P99 behavior is not obscured by averaging operations.
- Stability-Aware Optimization: We incorporate a stability-aware cost function into the DSE loop, ensuring that the

resulting GPU configurations are robust against tail latency spikes rather than merely optimized for average throughput.

We implemented the ChronosDSE framework on the open-source Vortex platform [21] and validated its effectiveness across multiple benchmarks. While our evaluation utilizes a RISC-V based GPGPU for reproducibility, the proposed stall-analysis methodology is ISA-agnostic and broadly applicable to commercial GPU architectures. Compared to baseline DSE approaches that rely on mean-based modeling, ChronosDSE improves long-tail performance by an average of 10% with only a negligible 2% trade-off in average throughput. Furthermore, leveraging our adaptive sampling strategy, it reduces the DSE exploration time by approximately 58% compared to exhaustive simulation, demonstrating its dual advantages in efficiency and robustness for data center inference.

## 2 BACKGROUND

Effective GPU architecture optimization requires bridging the gap between high-level service constraints and low-level hardware execution. This section first characterizes the primary optimization target, The tail latency challenge, and the standard methodology, GPU design space exploration. We then analyze why existing mean-based modeling fails to meet these targets, identifying the lack of temporal fidelity as a key gap. Finally, we introduce microarchitectural stall mechanisms as the essential physical granularity required to capture the missing dynamic behaviors.

**The Tail Latency Challenge.** As highlighted in the introduction, tail latency [10, 14, 16] has emerged as the primary constraint for modern data center services. While typically defined as the 95th to 99th percentile of end-to-end request latency (e.g., in Tail at Scale[10] and TailBench [16]), this system-level phenomenon originates from underlying hardware behaviors. In the specific context of GPU microarchitectures, tail latency manifests as execution intervals with extreme performance degradation. For example, benchmarks such as Lonestar6 [14] characterize this through the highest 5% high-variation intervals. Recent studies on large model inference (AMALI [3]) further demonstrate that relying solely on average metrics leads to error rates as high as 127.56%, confirming that masking these tail events results in modeling inaccuracies.

**GPU Design Space Exploration.** To optimize architectures for such stringent latency constraints, architects employ DSE [17, 19]. The design space of modern GPUs [6, 22] is expansive, characterized by non-linear relationships among Power, Performance, and Area (PPA) metrics. Navigating this space involves evaluating complex trade-offs to identify optimal parameters (e.g., cache sizes, scheduler policies). Given the prohibitive cost of exploring this space by RTL simulation, DSE frameworks must balance evaluation fidelity with simulation speed.

**Limitations of Mean-Based Modeling.** To accelerate this exploration, researchers have proposed various modeling approaches. Cycle-level attribution methods [2, 4] perform detailed analysis by simulation, while scaling-based methods [4, 15] rapidly predict performance using interval analysis or CPI stack scaling. Hybrid simulation methods [12, 24] employ multi-level abstraction to refine attribution. However, as noted in the introduction, these methods share a critical limitation: they rely heavily on average-behavior modeling. By aggregating performance metrics over long execution windows, they inadvertently smooth out the fine-grained temporal variability inherent in dynamic workloads like LLMs. Consequently, they fail to capture the transient latency spikes [3] that dictate P99 performance, leading to biased bottleneck identification.

**Microarchitectural Stall Mechanisms.** To overcome these limitations and capture the bottleneck migration described in the introduction, it is necessary to analyze execution dynamics at the finest granularity: the pipeline stall. Stall cycles represent the fundamental unit of performance loss. Common stall types include synchronization, idle, control, memory (data/structural), and compute (data/structural) stalls [24]. Collectively, analyzing the temporal distribution of these stall types provides a mechanism to distinguish between steady-state behavior and transient tail anomalies. We adopt this stall taxonomy as the foundational basis for our dynamic characterization.

## 3 ANALYSIS AND MOTIVATION

When GPUs were used primarily for graphics rendering or single HPC kernels, their workload patterns exhibited relative stability. Consequently, DSE relied on analysis tools focused on the average performance [2, 4, 15, 24]. However, as GPUs became general-purpose computing platforms, workload characteristics became highly complex. Performance bottlenecks are no longer determined solely by the average behavior, but are also increasingly dominated by tail latency effects (i.e., transient performance degradation).

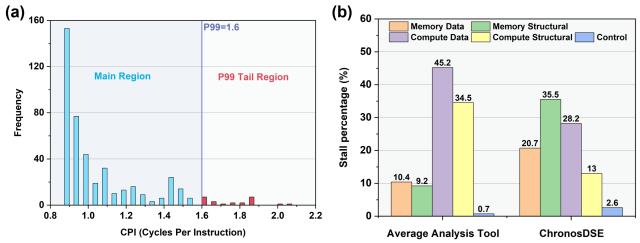



**Figure 2: (a) Long-Tail Characteristic in CPI Distribution; (b) Bottleneck Detection: mean-based vs. long-tail**

Using the lbm kernel as a study, we performed extensive interval sampling over one million execution cycles. As shown in Figure. 2(a), the CPI distribution exhibits a distinct long-tail characteristic. The tail region—comprising the top 1% of samples with the highest latency—demonstrates a significantly higher CPI than the average case, indicating the presence of severe transient bottlenecks.

Further analysis in Figure. 2(b) reveals a significant shift in the stall type composition within the tail region: the proportion of Memory Structural stall surges from 9.2% in the average domain to 35.5%, while Compute stalls almost disappears. This phenomenon aligns with findings from systems-level research [10, 14, 16], demonstrating that a single reliance on average performance metrics leads to severe performance inaccuracies.

To address this challenge, we propose ChronosDSE, a novel DSE tool. Its analytical capabilities extend beyond the average domain to specifically reveal the stall type distribution and characteristics within the tail latency domain.

# 4 DESIGN

Figure 3 illustrates the architectural workflow of ChronosDSE. The framework operates as a closed-loop iterative optimization system, taking architectural parameters and workloads as inputs to generate a Pareto-optimal frontier. The core evaluation engine (expanded on the right) integrates three methodological innovations that map directly to the subsequent sections.

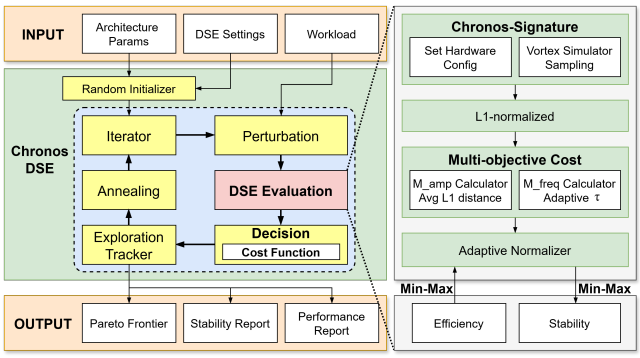



**Figure 3: ChronosDSE Architecture**

First, the **Chronos-Signature** module characterizes the temporal dynamics of bottleneck migration, transforming raw simulation traces into L1-normalized stability metrics.

Second, to ensure simulation efficiency, the profiling process employs an **Adaptive Window Sampling** strategy, which dynamically adjusts temporal resolution based on migration intensity.

Finally, the **Exploration** module implements a stability-aware exploration engine. It utilizes an adaptive normalizer and a multi-objective cost function to guide the Simulated Annealing process, shifting the optimization objective from static performance maximization to robust design.

## 4.1 Dynamic Stability Metric

Under complex workloads, system bottlenecks are not static; rather, they migrate continuously over time. Conventional DSE frameworks typically rely on average CPI or variance-based metrics to quantify performance fluctuations. However, these static metrics capture only aggregate behavior, failing to reveal the temporal evolution of bottlenecks.

We model the system execution process as a temporal trajectory. Within each sampling window, we aggregate the statistics of various stall types. To eliminate discrepancies in absolute counts, we construct an L1-normalized stall reason vector, denoted as $V(t)$, which represents the relative proportions of seven distinct stall categories at time window $t$:

$$V(t) = [\text{Sync}, \text{Idle}, \text{Control}, \text{Memory-data}, \text{Compute-data}, \text{Compute-structural}] \tag{1}$$

The resulting temporal trajectory intuitively characterizes the evolution of performance bottlenecks. Based on this trajectory, we formally define two types of dynamic stability metrics.

First, we introduce the **Migration Amplitude** ($M_{amp}$) to quantify the intensity of bottleneck state variations between adjacent time segments. Specifically, it is calculated by averaging the L1 distances between consecutive vectors:

$$M_{amp} = \frac{1}{N-1} \sum_{t=2}^{N} \|V(t) - V(t-1)\|_1 \tag{2}$$

where $N$ is the total number of sampling windows. This metric addresses the limitation of static profiling by quantifying the severity of microarchitectural state transitions, distinguishing between workloads with mild resource shifts and those with drastic contention oscillations.

## 4.2 Adaptive Window Sampling

The accuracy of stability metrics heavily depends on the sampling window size $W$. A fixed window size often fails to capture the multi-scale nature of bottleneck transients. To address this, ChronosDSE employs an adaptive sampling strategy that dynamically adjusts the window length based on the detected migration intensity.

We define the window update rule formally. Let $W_t$ denote the size of the sampling window in step $t$. The window for the next step, $W_{t+1}$, is adjusted as follows:

$$W_{t+1} = \begin{cases} \max(W_{min}, W_t/\gamma) & \text{if } M_{amp} \geq \tau \\ \min(W_{max}, W_t \cdot \gamma) & \text{if } M_{amp} < \tau \end{cases} \tag{3}$$

where $\gamma > 1$ is the window adjustment factor (set to 2 in our experiments). When significant migration is detected ($M_{amp} \geq \tau$), the window is shortened to enhance temporal resolution. Conversely, during stable phases, the window is lengthened to reduce simulation overhead. This mechanism ensures high-fidelity profiling for transient bottlenecks without prohibitive costs.

## 4.3 Stability-Aware Exploration

To overcome the limitations of mean-centric optimization, ChronosDSE shifts the DSE paradigm from solely minimizing average latency to a bi-objective optimization problem that explicitly trades off performance efficiency against microarchitectural stability.

**Formulation of the Stability-Aware Objective** Formally, let $\mathcal{S}$ denote the discrete architectural design space. For any candidate configuration $c \in \mathcal{S}$, we evaluate two distinct performance indicators: the average performance efficiency, denoted as $CPI_{avg}(c)$, and the bottleneck migration frequency, denoted as $M_{freq}(c)$. To navigate the trade-offs between these conflicting objectives, we construct a unified scalar cost function $\mathcal{J}(c, \mathcal{B})$.

A critical challenge in this formulation is the disparate magnitude and dynamic range of the two metrics. To address this, we employ an Online Min-Max normalization strategy. We define a dynamic observation boundary set $\mathcal{B} = \{CPI_{min}, CPI_{max}, M_{min}, M_{max}\}$, which tracks the extreme values observed throughout the exploration history. The cost function is thus parameterized by $\mathcal{B}$ as follows:

$$\mathcal{J}(c, \mathcal{B}) = \lambda \cdot \mathcal{N}_{eff}(c, \mathcal{B}) + (1-\lambda) \cdot \mathcal{N}_{stab}(c, \mathcal{B}) \tag{4}$$

where $\lambda \in [0, 1]$ represents the user's preference weight. The normalized terms $\mathcal{N}_{eff}$ and $\mathcal{N}_{stab}$ project the raw metrics onto a unified $[0, 1]$ scale:

$$\mathcal{N}_{eff}(c, \mathcal{B}) = \frac{CPI_{avg}(c) - CPI_{min}}{CPI_{max} - CPI_{min}} \tag{5}$$

**Algorithm 1** ChronosDSE Algorithm

**Require:** Design Space $\mathcal{S}$, Cooling Schedule $\langle T_0, \alpha, T_{\min} \rangle$, Preference $\lambda$
**Ensure:** Robust Configuration $c^*$

$c \leftarrow \textsc{UniformSample}(\mathcal{S})$
$\mathcal{B} \leftarrow \textsc{InitRange}(\textsc{Simulate}(c))$ ▷ Initialize min-max bounds $\mathcal{B}$
$c^* \leftarrow c$; $T \leftarrow T_0$

**while** $T > T_{\min}$ **do**
  $c' \leftarrow \textsc{Neighbor}(c)$ ▷ Perturbation in discrete space
  $\mathbf{m}' \leftarrow \textsc{Simulate}(c')$ ▷ Get metrics $\langle CPI_{\text{avg}}, M_{\text{freq}} \rangle$
  $\mathcal{B} \leftarrow \textsc{UpdateRange}(\mathcal{B}, \mathbf{m}')$ ▷ Online normalization update
  $\Delta\mathcal{J} \leftarrow \mathcal{J}(c', \mathcal{B}) - \mathcal{J}(c, \mathcal{B})$
  **Metropolis Criterion:**
  **if** $\Delta\mathcal{J} < 0 \vee \text{Random}(0,1) < e^{-\Delta\mathcal{J}/T}$ **then**
    $c \leftarrow c'$
    **if** $\mathcal{J}(c, \mathcal{B}) < \mathcal{J}(c^*, \mathcal{B})$ **then**
      $c^* \leftarrow c$ ▷ Update best solution
  $T \leftarrow \alpha \cdot T$
**return** $c^*$

$$\mathcal{N}_{stab}(c, \mathcal{B}) = \frac{M_{freq}(c) - M_{min}}{M_{max} - M_{min}} \tag{6}$$

**Robust Search via Cost Re-alignment** We employ a Simulated Annealing (SA) framework to traverse the non-convex design space $\mathcal{S}$. However, the integration of online normalization introduces a non-stationary objective landscape: as the exploration discovers new outliers, the boundary set $\mathcal{B}$ expands, rendering the cost values calculated in previous iterations invalid for direct comparison.

Second, we define the **Migration Frequency** ($M_{freq}$), which counts the occurrences of significant switching events:

$$M_{freq} = \sum_{t=2}^{N} \mathbb{I}(\Delta(t) \geq \tau) \tag{7}$$

where $\Delta(t) = \|V(t) - V(t-1)\|_1$ denotes the instantaneous change. To robustly distinguish significant shifts from noise, we employ an adaptive threshold $\tau$:

$$\tau = \alpha \cdot \text{median}(\{\Delta(t)\}_{t=1}^{N}) \tag{8}$$

Here, $\alpha$ is a sensitivity hyperparameter (typically set to $\alpha = 1.5$), and $\mathbb{I}(\cdot)$ is the indicator function. Unlike variance which only reflects global dispersion, $M_{freq}$ explicitly captures the temporal instability of the execution, serving as a direct proxy for the transient latency spikes that jeopardize tail latency targets.

To ensure mathematical rigor and convergence, ChronosDSE enforces a Cost Re-alignment mechanism. Unlike traditional SA which relies on static cost differentials, our approach aligns the baseline before every Metropolis evaluation. Specifically, when transitioning from a current state $c$ to a neighbor candidate $c'$, we first update the boundary set $\mathcal{B} \to \mathcal{B}'$ using the metrics of $c'$.

Crucially, we then re-evaluate the cost of the current state $c$ under the new boundaries $\mathcal{B}'$. The cost difference $\Delta\mathcal{J}$ is computed as:

$$\Delta\mathcal{J} = \mathcal{J}(c', \mathcal{B}') - \mathcal{J}(c, \mathcal{B}') \tag{9}$$

This mechanism ensures that the acceptance probability $P$ is calculated as:

$$P = \min\left(1, \exp\left(-\frac{\Delta\mathcal{J}}{T}\right)\right) \tag{10}$$

This derivation ensures a fair comparison on a consistent scale. Furthermore, since the boundary expansion affects global optimality, the incumbent best solution $c^*$ is also explicitly re-evaluated against the current candidate under the updated normalization bounds $\mathcal{B}'$. This guarantees that $c^*$ remains the valid global optimum within the current observable space. The complete workflow is detailed in Algorithm 1.

# 5 EVALUATION

In this section, we present a systematic evaluation of the proposed ChronosDSE framework, addressing three core research questions:

(1) Validity: Does the traditional $CPI_{avg}$ metric obscure critical performance anomalies, and can Chronos-Signature effectively uncover these blind spots?

(2) Efficiency & Robustness: Does ChronosDSE outperform Baseline-DSE in terms of both search efficiency and solution quality (i.e., tail robustness)?

(3) Interpretability: How do the architectural resource allocations identified by ChronosDSE differ from those of the baseline, and how do these decisions mitigate microarchitectural contention?

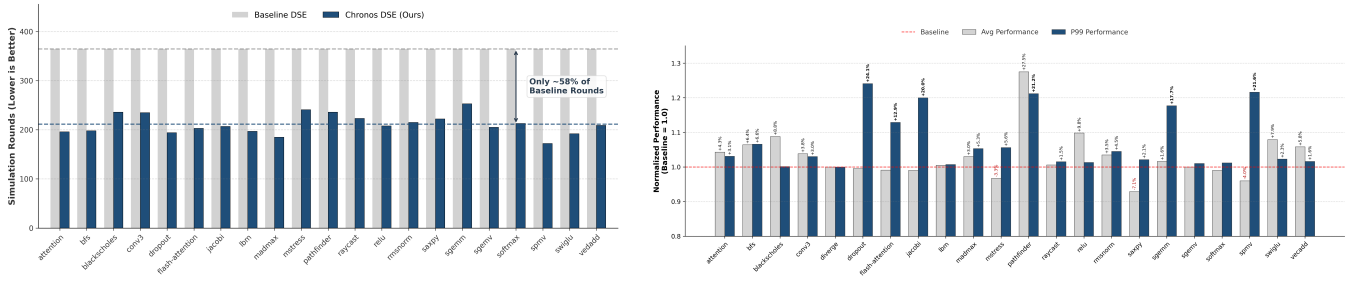
## 5.1 Experimental Setup

We employ the Vortex GPGPU simulator as our primary experimental platform. Supporting the modern RISC-V GPU ISA, Vortex features a lightweight and modular architecture that facilitates the implementation of the high-frequency sampler for Chronos-Signature. To ensure fairness and comparability, all comparative experiments—between Baseline-DSE and ChronosDSE—are conducted on this unified platform. We selected a diverse set of benchmarks covering deep learning, linear algebra, scientific computing, graph algorithms, and stress tests, as detailed in Table. 1 .

**Table 1: Workloads used for evaluation**

| Category | Workloads |
|---|---|
| LLM Operator | attention[23], flash-attention[7], softmax, dropout, swiglu, relu, conv3, rmsnorm |
| Linear Algebra | sgemv, sgemm, saxpy, spmv, vecadd |
| Scientific Computing | jacobi[20], stencil3d[9], lbm[18], gaussian[5] |
| Graph Algorithms | bfs, pathfinder, raycast, diverge |
| Stress Test | blackscholes, madmax, mstress |

Our design space is defined by a set of critical GPU microarchitectural parameters, encompassing memory hierarchy, computational resources, and scheduling policies. Specifically, these parameters include cache size and associativity, shared memory capacity, the number of memory controllers, Streaming Multiprocessor (SM) count, register file size, and scheduler queue depth. Collectively, these parameters dictate the system's PPA characteristics. For clarity, Table. 2 details the primary parameters and their respective




**(a) Number of Evaluations to Convergence.** **(b) Comparison of $CPI_{avg}$ and $CPI_{p99}$ across Benchmarks.**

**Figure 4: Efficiency and Performance Analysis: (a) Convergence Speed (Number of Evaluations to Convergence) and (b) CPI Optimization (Comparison of $CPI_{avg}$ and $CPI_{p99}$ across Benchmarks).**

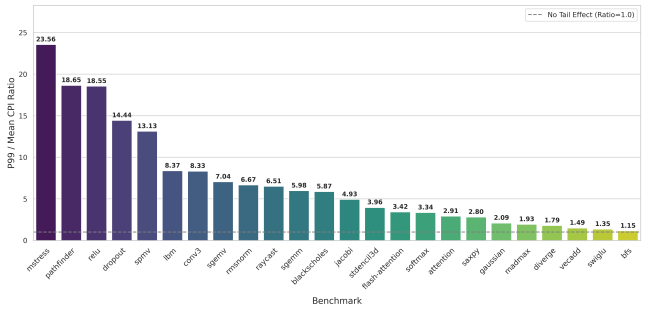



**Figure 5: Comparison of P99-to-Mean CPI for Benchmarks**

value ranges. To ensure a fair comparison across different methods, all explorations are conducted under a unified maximum hardware configuration and a fixed simulation budget.

## 5.2 Validity of Chronos-Signature

We first scrutinize the limitations of traditional static metrics $CPI_{avg}$ in characterizing real-world performance anomalies. As illustrated in Figure. 5 , the p99-to-Mean CPI ratios for benchmarks such as *pathfinder*, *relu*, and *dropout* reveal significant heavy-tailed behavior, with $CPI_{p99}$ reaching 10–18 *times* the average. This observation highlights that architectures with acceptable average performance often suffer from severe runtime jitter. Specifically, $CPI_{avg}$ aggregates performance over the entire execution timeline, effectively masking fine-grained temporal fluctuations. Consequently, optimizers guided solely by $CPI_{avg}$ risk converging to configurations that are statistically efficient but inherently unstable.

To demonstrate that Chronos-Signature circumvents this blind spot, we investigate the correlation between tail latency and bottleneck migration. We construct a regression model with $CPI_{p99}$ as the dependent variable (shown in Figure. 6). The results reveal that $CPI_{avg}$ alone lacks explanatory power ($R^2 = 0.342$), indicating that tail latency degradation cannot be attributed solely to low overall efficiency. Conversely, incorporating the migration frequency ($M_{freq}$) significantly boosts the $R^2$ to 0.534. This corroborates that frequent bottleneck switching is the primary culprit behind performance unpredictability—where each switch incurs pipeline draining or cache thrashing. By combining $M_{amp}$ and $M_{freq}$ , the model's accuracy further improves to 0.551. These findings validate that Chronos-Signature successfully quantifies the temporal evolution of bottlenecks, providing a high-dimensional metric independent of CPI to identify and prune high-risk designs.

**Table 2: Microarchitecture design space specification**

| Component | Description | Values | # |
|---|---|---|---|
| | Scaling | | |
| Clusters | number of clusters | 1:4:1 | 4 |
| Cores | number of cores per cluster | 1:4:1 | 4 |
| Warps | number of warps per core | 4, 8, 16, 32 | 4 |
| Threads | number of threads per warp | 4, 8, 16, 32 | 4 |
| | Pipeline | | |
| Issue Width | instructions issued/cycle | 1:4:1 | 4 |
| IBuffer | instruction buffer entries | 4:32:4 | 8 |
| LSU Queue | load/store queue entries | 4:32:4 | 8 |
| | Units | | |
| LSU | number of load/store units | 1:4:1 | 4 |
| SFU | number of SFUs | 1:4:1 | 4 |
| ALU | number of ALUs | 1:4:1 | 4 |
| FPU | number of FPUs | 1:4:1 | 4 |
| Scheduler | warp scheduler policy | RR, GTO, TMASK | 3 |
| GPR Banks | number of GPR banks | 4, 8, 16, 32 | 4 |
| | Memory | | |
| I$ Size | I-cache size (KB) | 16, 32, 64 | 3 |
| I$ Assoc. | I-cache associativity | 2, 4, 8 | 3 |
| I$ MSHR | number of I-cache MSHRs | 4:32:4 | 8 |
| D$ Size | D-cache size (KB) | 16, 32, 64 | 3 |
| D$ Assoc. | D-cache associativity | 2, 4, 8 | 3 |
| D$ MSHR | number of MSHRs | 4:32:4 | 8 |
| **Total Size** | $6.44 \times 10^9$ | | |

## 5.3 Efficiency and Robustness Comparison

We perform a comprehensive DSE using the ChronosDSE framework. Regarding search efficiency, Figure. 6 shows that ChronosDSE achieves significant acceleration, converging to the Pareto frontier with merely 58% of the simulation budget required by the baseline. This efficiency gain is attributed to the early pruning capability of our stability metrics: while Baseline-DSE oscillates within local optima characterized by low $CPI_{avg}$ but high instability, ChronosDSE leverages the stability penalty ($N_{stab}$) to swiftly filter out these

volatile regions, focusing the limited simulation budget on the robust-and-efficient subspace.

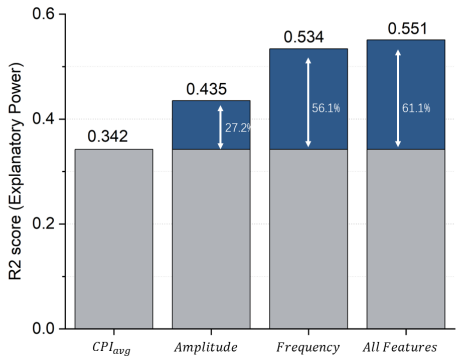



**Figure 6: Impact of Integrating Chronos-Signature Metrics on Modeling Tail Performance.**

In terms of solution quality (4b), ChronosDSE attains Pareto improvements while significantly enhancing robustness. Notably, for timing-sensitive workloads like dropout, lbm, and spmv, the architectures identified by ChronosDSE improve $CPI_{p99}$ by 24.1%, 20.0%, and 21.6%, respectively. This demonstrates that integrating dynamic stability into the optimization objective translates directly into better QoS. Furthermore, this robustness is not achieved at the expense of efficiency; most benchmarks exhibit normalized scores superior to the baseline (>1.0). This indicates that by mitigating resource contention and frequent bottleneck shifts, the system achieves smoother execution, which in turn benefits overall throughput.

## 5.4 Architectural Decision Analysis

To uncover the microarchitectural mechanisms behind ChronosDSE's advantages, we dissect the architectural decisions using the dropout benchmark as a case study in Figure. 7.

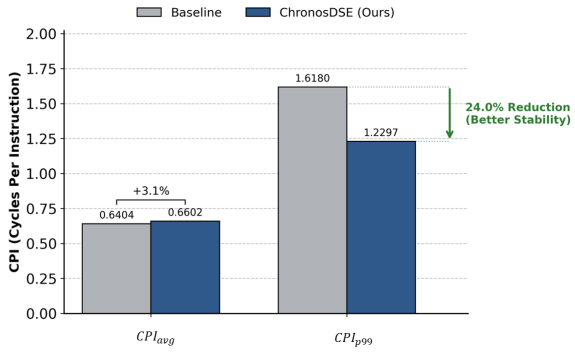



**Figure 7: Case Study on Dropout Benchmark**

Baseline-DSE favors an aggressive resource allocation strategy, selecting a high-concurrency configuration (16 warps/core, 16 MSHR entries, GTO scheduling). Theoretically, abundant warps and GTO scheduling should hide latency, yielding a lower $CPI_{avg}$ (0.6404). However, in the actual microexecution, this excessive parallelism triggers severe L1 D-Cache thrashing. As defined, this contention forces the bottleneck to oscillate violently between Compute_Data and Memory_Structural stall, degrading $CPI_{p99}$ to 1.6180.

In contrast, ChronosDSE identifies these hidden contention risks and converges to a balanced configuration (8 warps/core, 8 MSHR entries, Round-Robin scheduling). By reclaiming budget from over-provisioned resources (reducing warp count) and adopting a fairer Round-Robin scheduling, ChronosDSE effectively alleviates resource pressure. Although this incurs a marginal 3% increase in $CPI_{avg}$ (0.6602), it drastically reduces bottleneck migration frequency, suppressing $CPI_{p99}$ from 1.6180 to 1.2297 (about 24% optimization). This case study confirms that ChronosDSE looks beyond static averages to strike an optimal balance between resource utilization and contention, guiding the DSE toward architectures that remain resilient under complex, real-world loads.

## 6 CONCLUSION

This paper addresses the critical fidelity gap in designing GPGPU architectures for dynamic LLM inference, where traditional mean-based metrics fail to capture essential service quality constraints. We propose ChronosDSE, which pioneers the integration of system-level tail latency metrics into the microarchitectural GPGPU Design Space Exploration loop. Unlike prior approaches that rely on static averaging, ChronosDSE shifts the paradigm to dynamic characterization via *Chronos-Signature* and adaptive sampling, explicitly identifying transient bottlenecks that were previously invisible. Our evaluation confirms that this approach improves p99 performance by 10% while reducing exploration overhead by 58%. These findings offer a scalable path for next-generation infrastructure, with future potential to extend into chiplet-based architectures and runtime stability management.

## References


[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, Alexey Tumanov, and Ramachandran Ramjee. 2024. Taming throughput-latency tradeoff in LLM inference with sarathi-serve. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 7, 18 pages.

[2] Johnathan Alsop, Matthew D. Sinclair, Rakesh Komuravelli, and Sarita V. Adve. 2016. GSI: A GPU Stall Inspector to characterize the sources of memory stalls for tightly coupled GPUs. In *2016 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 172–182. doi:10.1109/ISPASS.2016.7482092

[3] Shiheng Cao, Junmin Wu, Junshi Chen, Hong An, and Zhibin Yu. 2025. AMALI: An Analytical Model for Accurately Modeling LLM Inference on Modern GPUs. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1495–1508. doi:10.1145/3695053.3731064

[4] Hanna Cha, Sungchul Lee, Jounghoo Lee, Yeonan Ha, Joonsung Kim, and Youngsok Kim. 2025. GCStack+GCScaler: Fast and Accurate GPU Performance Analyses Using Fine-Grained Stall Cycle Accounting and Interval Analysis. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1509–1523. doi:10.1145/3695053.3731068

[5] Shuai Che, Michael Boyer, Jiayuan Meng, David Tarjan, Jeremy W. Sheaffer, Sang-Ha Lee, and Kevin Skadron. 2009. Rodinia: A benchmark suite for heterogeneous computing. In *2009 IEEE International Symposium on Workload Characterization (IISWC)*. 44–54. doi:10.1109/IISWC.2009.5306797

[6] Jack Choquette, Wishwesh Gandhi, Olivier Giroux, Nick Stam, and Ronny Krashinsky. 2021. NVIDIA A100 Tensor Core GPU: Performance and Innovation. *IEEE Micro* 41, 2 (2021), 29–35. doi:10.1109/MM.2021.3061394

[7] Tri Dao. 2022. *FLASHATTENTION: fast and memory-efficient exact attention with IO-awareness.* arXiv:2205.14135 doi:10.48550/arXiv.2205.14135

[8] Tri Dao. 2024. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.* arXiv:2307.08691 doi:10.48550/arXiv.2307.08691

[9] Kaushik Datta, Mark Murphy, Vasily Volkov, Samuel Williams, Jonathan Carter, Leonid Oliker, David Patterson, John Shalf, and Katherine Yelick. 2008. Stencil computation optimization and auto-tuning on state-of-the-art multicore architectures. In *SC '08: Proceedings of the 2008 ACM/IEEE Conference on Supercomputing*. 1–12. doi:10.1109/SC.2008.5222004

[10] Jeffrey Dean and Luiz André Barroso. 2013. The tail at scale. *Commun. ACM* 56, 2 (Feb. 2013), 74–80. doi:10.1145/2408776.2408794

[11] Simon Gilardi, Qirong Yuan, Davis Blalock, Dimitris Pal, and Davis Blalock. 2023. LLMPerf: A Benchmark for Evaluating Large Language Model Inference Services. (2023). arXiv:2310.19536 doi:10.48550/arXiv.2503.11244

[12] Mahmoud Khairy, Zhesheng Shen, Tor M. Aamodt, and Timothy G. Rogers. 2020. Accel-sim: an extensible simulation framework for validated GPU modeling. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture* (Virtual Event) *(ISCA '20)*. IEEE Press, 473—-486. doi:10.1109/ISCA45697.2020.00047

[13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles* (Koblenz, Germany) *(SOSP '23)*. Association for Computing Machinery, New York, NY, USA, 611–626. doi:10.1145/3600006.3613165

[14] Ankur Lahiry, Ayush Pokharel, Seth Ockerman, Amal Gueroudji, Line Pouchard, and Tanzima Z. Islam. 2025. *Scalable GPU Performance Variability Analysis framework.* arXiv:2506.20674 doi:10.48550/arXiv.2506.20674

[15] Jounghoo Lee, Yeonan Ha, Suhyun Lee, Jinyoung Woo, Jinho Lee, Hanhwi Jang, and Youngsok Kim. 2022. GCoM: a detailed GPU core model for accurate analytical modeling of modern GPUs. In *Proceedings of the 49th Annual International Symposium on Computer Architecture* (New York, New York) *(ISCA '22)*. Association for Computing Machinery, New York, NY, USA, 424–436. doi:10.1145/3470496.3527384

[16] Zhilin Li, Lucia Pons, Salvador Petit, Julio Sahuquillo, and Julio Pons. 2025. *TailBench++: Flexible Multi-Client, Multi-Server Benchmarking for Latency-Critical Workloads.* arXiv:2505.03600 doi:10.48550/arXiv.2505.03600

[17] Arash Nasr-Esfahany, Mohammad Alizadeh, Victor Lee, Hanna Alam, Brett W. Coon, David Culler, Vidushi Dadu, Martin Dixon, Henry M. Levy, Santosh Pandey, Parthasarathy Ranganathan, and Amir Yazdanbakhsh. 2025. Concorde: Fast and Accurate CPU Performance Modeling with Compositional Analytical-ML Fusion. In *Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25)*. Association for Computing Machinery, New York, NY, USA, 1480–1494. doi:10.1145/3695053.3731037

[18] Y. H. Qian, D. D'Humières, and P. Lallemand. 1992. Lattice BGK Models for Navier-Stokes Equation. *Europhysics Letters* 17, 6 (feb 1992), 479. doi:10.1209/0295-5075/17/6/001

[19] Ling Ren, Xiangyao Yu, Christopher W. Fletcher, Marten van Dijk, and Srinivas Devadas. 2013. Design space exploration and optimization of path oblivious RAM in secure processors. In *Proceedings of the 40th Annual International Symposium on Computer Architecture* (Tel-Aviv, Israel) *(ISCA '13)*. Association for Computing Machinery, New York, NY, USA, 571—-582. doi:10.1145/2485922.2485971

[20] Y. Saad. 2003. *Iterative Methods for Sparse Linear Systems* (2nd ed.). Society for Industrial and Applied Mathematics, USA.

[21] Blaise Tine, Krishna Praveen Yalamarthy, Fares Elsabbagh, and Kim Hyesoon. 2021. Vortex: Extending the RISC-V ISA for GPGPU and 3D-Graphics. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture* (Virtual Event, Greece) *(MICRO '21)*. Association for Computing Machinery, New York, NY, USA, 754–766. doi:10.1145/3466752.3480128

[22] Ajay Tirumala and Raymond Wong. 2024. NVIDIA Blackwell Platform: Advancing Generative AI and Accelerated Computing. In *2024 IEEE Hot Chips 36 Symposium (HCS)*. 1–33. doi:10.1109/HCS61935.2024.10665247

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. *Attention is all you need.* arXiv:1706.03762 doi:10.48550/arXiv.1706.03762

[24] Jianchao Yang, Mei Wen, Dong Chen, Zhaoyun Chen, Zeyu Xue, Yuhang Li, Junzhong Shen, and Yang Shi. 2024. HyFiSS: A Hybrid Fidelity Stall-Aware Simulator for GPGPUs. In *Proceedings of the 2024 57th IEEE/ACM International Symposium on Microarchitecture* (Austin, TX, USA) *(MICRO '24)*. IEEE Press, 168–185. doi:10.1109/MICRO61859.2024.00022

[25] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 135–154.